

FB Tapeswap v2 ISA:

Op	Code	Action	Inverse
<	1	CL -= 1	>
>	2	CL += 1	<
{	3	H0 -= 1	}
}	4	H0 += 1	{
(	5	H1 -= 1	)
)	6	H1 += 1	(
+	7	tape[H0] += 1	-
-	8	tape[H0] -= 1	+
x	9	swap(tape[H0], tape[H1])	self
.	10	tape[H0] += tape[H1]	,
,	14	tape[H0] -= tape[H1]	.
J	11	if tape[CL] 0: tapeswap jump to tape[CL]	self
S	12	swap(tape[CL], tape[H0])	self
G	13	swap(CL, tape[H0])	self

v2 J details: Condition AND target are both **tape[CL]**

```

if tape[CL] != 0:
    target = tape[CL]
    swap(tape[IP], tape[target]) # move J to target location
    swap(IP, tape[CL])          # swap addresses
IP += 1

```

FB Tapeswap v3 ISA:

Op	Code	Action	Inverse
<	1	CL -= 1	>
>	2	CL += 1	<
{	3	H0 -= 1	}
}	4	H0 += 1	{
(	5	H1 -= 1	)
)	6	H1 += 1	(
+	7	tape[H0] += 1	-
-	8	tape[H0] -= 1	+
x	9	swap(tape[H0], tape[H1])	self
.	10	tape[H0] += tape[H1]	,
,	14	tape[H0] -= tape[H1]	.
J	11	if tape[CL] 0: tapeswap jump to tape[H1]	self
S	12	swap(tape[CL], tape[H0])	self
G	13	swap(CL, tape[H0])	self

v3 J details: Condition is `tape[CL]`, target is `tape[H1]` (separated!)

```
if tape[CL] != 0:
    target = tape[H1]
    swap(tape[IP], tape[target]) # move J to target location
    swap(IP, tape[H1])          # swap addresses (puts return addr in tape[H1])
IP += 1
```

Key difference:

v2: `tape[CL]` is both condition and target (problematic for loops where you want condition=counter, target=loop address)

v3: `tape[CL]` is condition, `tape[H1]` is target (separated, but target gets overwritten with return address after first jump)

Both do the tapeswap (`swap(tape[IP], tape[target])`) which moves J around on the tape, breaking loops that revisit code.